2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
Daejeon Convention Center
October 9-14, 2016, Daejeon, Korea

One-Shot Learning of Manipulation Skills with Online Dynamics Adaptation and Neural Network Priors

Justin Fu, Sergey Levine, Pieter Abbeel

***Abstract*— One of the key challenges in applying reinforcement learning to complex robotic control tasks is the need to gather large amounts of experience in order to find an effective policy for the task at hand. Model-based reinforcement learning can achieve good sample efficiency, but requires the ability to learn a model of the dynamics that is good enough to learn an effective policy. In this work, we develop a model-based reinforcement learning algorithm that combines prior knowledge from previous tasks with online adaptation of the dynamics model. These two ingredients enable highly sample-efficient learning even in regimes where estimating the true dynamics is very difficult, since the online model adaptation allows the method to locally compensate for unmodeled variation in the dynamics. We encode the prior experience into a neural network dynamics model, adapt it online by progressively refitting a local linear model of the dynamics, and use model predictive control to plan under these dynamics. Our experimental results show that this approach can be used to solve a variety of complex robotic manipulation tasks in just a single attempt, using prior data from other manipulation behaviors.**

I. Introduction

One of the remarkable features of human and animal motor control is the ability to quickly adapt to new situations. When a child is asked, for example, to stack two unfamiliar Lego blocks, he or she might play with the objects and take a small amount of time to learn about their dynamics, but will typically succeed at the task very quickly. This is sometimes referred to as one-shot learning, where an agent must successfully perform a task given one, or very few attempts. Reinforcement learning (RL) provides a computational framework for robots to learn new motor skills, but typical applications of RL focus more on mastery than one-shot learning, and require a substantial number of training episodes [1]. Model-based RL methods reduce the required interaction time by acquiring a model of the system dynamics, and using this model to discover an effective policy. However, model-based RL algorithms that operate in this way must be equipped with a model that can represent a good approximation to the dynamics, and they must be provided with sufficient experience to optimize this model to produce accurate dynamics predictions [2]. A single attempt at the task is often insufficient to obtain such an accurate model, and while these challenges can be mitigated by incorporating domain knowledge about the system [3], [4] or demonstrations [5], they make the development of a general-purpose one-shot learning method difficult.

We propose to address the first challenge by developing a method that can use a coarse model of the system dynamics

Department of Electrical Engineering and Computer Science, University of California, Berkeley, Berkeley, CA 94709

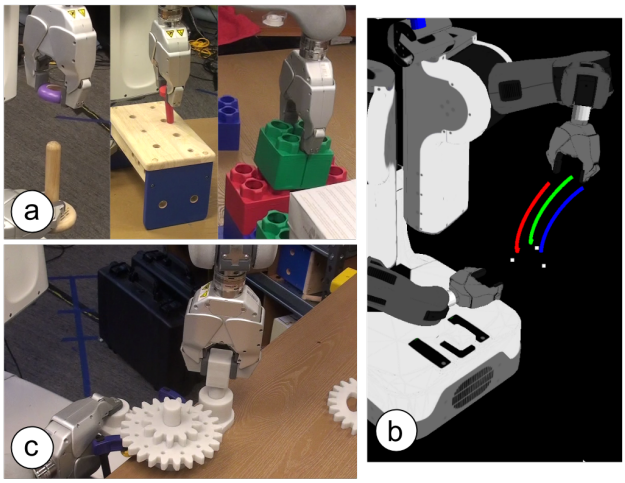

Fig. 1: Diagram of our method: the robot uses prior experience from other tasks (a) to fit a neural network model of the dynamics of object interaction tasks (b). When faced with a new task (c), our algorithm learns a new model online during task execution, using the neural network as a prior. This new model is used to plan actions, allowing for one-shot learning of new skills.

that is adapted online to better match the most recent experience. This frees the method from needing a representation that can capture global dynamics with high accuracy, and requires only the ability to accurately represent the dynamics locally, which we show can be done with a simple linear model. The accuracy of the global model still affects the proficiency with which a new task can be performed, since more accurate models require less adaptation, but even an inaccurate initial model is sufficient for our method to perform a variety of manipulation tasks on the first attempt. We show that even a simple linear global model can allow this framework to perform a variety of manipulation tasks with nonlinear dynamics, while a more sophisticated nonlinear model based on neural networks greatly improves the success rate on more complex tasks. Online adaptation also offers us a way to address the second challenge: since the global model does not need to be very accurate, it can be estimated using data from tasks that are different from the one being attempted. This enables our approach to perform one-shot learning of new tasks by using the robot's prior experience on *other* tasks, without requiring explicit domain knowledge or demonstrations to be provided by the designer. A diagram of our method is shown in Figure 1. We demonstrate our approach by learning a variety of challenging, contact-rich manipulation behaviors on a PR2 robot. All of our motion skills involve low-level torque control of the robot's motors, and high dimensional state, corresponding to joint angles,

joint velocities, and end-effector pose.

II. Related Work

Reinforcement learning techniques have been applied to a range of robotic control problems [2]. Such methods can be broadly categorized as model-free methods, which directly learn a control policy from system interaction [6], and model-based methods, which first learn a model of the system dynamics, and then optimize the policy under this model [1]. Model-based methods can be substantially more sample-efficient and typically achieve the fastest learning times, but they require a model representation that can be used to learn an accurate estimate of the true dynamics. Typically, in order to learn the model from a small amount of interaction, they make various assumptions, such as smoothness [7], [8] or access to prior knowledge about the system [3], [4], since the general problem of learning a complex, nonlinear function from a small amount of data is difficult. Contact-rich robotic manipulation skills present an especially challenging model learning problem, and simple assumptions, such as smoothness and prior knowledge, are often insufficient to acquire a model that is accurate enough for manipulation.

In this paper, we instead specifically seek to learn new behaviors with a coarse model of the system dynamics that does not adequately capture all of the intricacies of the real system, and locally adapt this model online based on the most recent experience. In this way, our approach resembles adaptive control [9]. However, unlike standard adaptive control, we incorporate an expressive (but potentially inaccurate) prior model of the dynamics, constructed from previous experience on other manipulation tasks, and use only high-level objectives, such as the desired position of a target object, rather than simple trajectory tracking. This makes our method suitable for complex robotic manipulation skills with only high-level specification in the form of a cost function. Experience-based priors have previously been suggested in reinforcement learning [10], though typically in the context of accelerating iterative model-free learning. In contrast, our work demonstrates one-shot learning, where the robot can immediately perform new tasks by leveraging prior experience.

In order to choose the actions under our locally adapted model, we use model predictive control (MPC) based on differential dynamic programming (DDP) [11]. Neural network models have recently been combined with MPC for planar task-space control of cutting tasks [12]. While this approach is similar in spirit to the one presented in this work, we tackle a wider range of robotic manipulation tasks using direct, low-level torque control of the entire 7 degree of freedom arm. Since the tasks that we tackle are quite different from the prior experience used to train the neural network, local adaptation is required for success, as shown in our experimental results. Outside of robotic manipulation, MPC has been combined with online and offline adaptation [13], [14], [15], but typically in the context of trajectory tracking, rather than learning new skills with high-level specification.

III. Background

In reinforcement learning, as well as in optimal control, the aim is to control a dynamical system given by $\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t)$ by choosing the actions $\mathbf{u}_t$ to minimize the total cost $\sum_{t=1}^{T} \ell(\mathbf{x}_t, \mathbf{u}_t)$, where T is the time horizon. We consider fixed-horizon episodic tasks in this paper, though our method can also be extended to an infinite horizon formulation. When the system dynamics $f(\mathbf{x}_t, \mathbf{u}_t)$ are unknown, we can construct an estimate of the dynamics $\hat{f}(\mathbf{x}_t, \mathbf{u}_t)$, for example by fitting a parameterized function approximator to data from prior system interactions, and then optimize a sequence of actions under the estimated dynamics $\hat{f}(\mathbf{x}_t, \mathbf{u}_t)$.

We will make use of the iterative linear quadratic regulator (iLQR) algorithm [16] to optimize the actions with respect to the cost under an estimated dynamics model. This algorithm can be viewed as an application of the Gauss-Newton method for trajectory optimization, and requires iteratively linearizing the dynamics around the current nominal trajectory, denoted $\hat{\tau} = \{\hat{\mathbf{x}}_1, \hat{\mathbf{u}}_1, \ldots, \hat{\mathbf{x}}_T, \hat{\mathbf{u}}_T\}$, constructing a quadratic approximation to the cost, computing the optimal actions with respect to this approximation of the dynamics and cost, and running the resulting actions forward to obtain a new nominal trajectory. In the derivation below, we will assume that the linearized stochastic dynamics are given by $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t) = \mathcal{N}(f_{\mathbf{x}t}\mathbf{x}_t + f_{\mathbf{u}t}\mathbf{u}_t + f_{ct}, \mathbf{F}_t)$, where $\mathbf{F}_t$ is the covariance of the Gaussian dynamics noise, and the quadratic approximation to the cost consists of a linear term $\ell_{\mathbf{xu}t}$ and a quadratic term $\ell_{\mathbf{xu},\mathbf{xu}t}$, where the subscripts denote differentiation with respect to the vector $[\mathbf{x}_t; \mathbf{u}_t]$. When the dynamics are linear and the cost is quadratic, the Q-function and the value function are both quadratic, and given by

$$V(\mathbf{x}_t) = \frac{1}{2}\mathbf{x}_t^{\mathrm{T}} V_{\mathbf{x},\mathbf{x}t}\mathbf{x}_t + \mathbf{x}_t^{\mathrm{T}} V_{\mathbf{x}t} + \text{const}$$

$$Q(\mathbf{x}_t, \mathbf{u}_t) = \frac{1}{2}[\mathbf{x}_t;\mathbf{u}_t]^{\mathrm{T}} Q_{\mathbf{xu},\mathbf{xu}t}[\mathbf{x}_t;\mathbf{u}_t] + [\mathbf{x}_t;\mathbf{u}_t]^{\mathrm{T}} Q_{\mathbf{xu}t} + \text{const}$$

We can express them with the following recurrence:

$$\begin{aligned} Q_{\mathbf{xu},\mathbf{xu}t} &= \ell_{\mathbf{xu},\mathbf{xu}t} + \gamma f_{\mathbf{xu}t}^{\mathrm{T}} V_{\mathbf{x},\mathbf{x}t+1} f_{\mathbf{xu}t} \\ Q_{\mathbf{xu}t} &= \ell_{\mathbf{xu}t} + \gamma f_{\mathbf{xu}t}^{\mathrm{T}} V_{\mathbf{x}t+1} \\ V_{\mathbf{x},\mathbf{x}t} &= Q_{\mathbf{x},\mathbf{x}t} - Q_{\mathbf{u},\mathbf{x}t}^{\mathrm{T}} Q_{\mathbf{u},\mathbf{u}t}^{-1} Q_{\mathbf{u},\mathbf{x}t} \\ V_{\mathbf{x}t} &= Q_{\mathbf{x}t} - Q_{\mathbf{u},\mathbf{x}t}^{\mathrm{T}} Q_{\mathbf{u},\mathbf{u}t}^{-1} Q_{\mathbf{u}t}, \end{aligned}$$

which allows us to compute the optimal control law as $g(\mathbf{x}_t) = \hat{\mathbf{u}}_t + \mathbf{k}_t + \mathbf{K}_t(\mathbf{x}_t - \hat{\mathbf{x}}_t)$, where $\mathbf{K}_t = -Q_{\mathbf{u},\mathbf{u}t}^{-1} Q_{\mathbf{u},\mathbf{x}t}$ and $\mathbf{k}_t = -Q_{\mathbf{u},\mathbf{u}t}^{-1} Q_{\mathbf{u}t}$. Performing a forward rollout using this control law allows us to find a new nominal trajectory, and the backward dynamic programming pass is repeated around this trajectory to take the next Gauss-Newton step. The discount factor γ allows us to reduce the weight on later states. While DDP is typically used with $\gamma = 1$, we found that we could obtain better results under uncertain, estimated dynamics with $\gamma = 0.95$. Intuitively, this reflects the fact that predictions further into the future are less likely to correspond to reality under uncertain dynamics, so it makes sense not to weight them as highly.

Iterative LQR can be used to optimize a trajectory offline under known system dynamics, or even with dynamics estimated from previous executions on a physical system [17]. However, in this work we instead use iterative LQR to perform model-predictive control, where the policy is recomputed in real time at each time step to update the next action in response to the current state. Iterative LQR is well suited for MPC because it is fast, can be effectively used with short horizons, and readily allows warm-starting with the previous solution [18].

However, using iterative LQR for MPC still requires us to obtain an estimate of the linearized system dynamics, given by $f_{\mathbf{x}t}$ and $f_{\mathbf{u}t}$. The dynamics can be estimated from a simulator of the physical system, but this requires considerable knowledge about the system being controlled. In the case of robotic manipulation, it is reasonable to expect to obtain a good model of the robot, but we often lack a model of the objects that the robot is interacting with, so being able to handle unknown dynamics is highly desirable. In Section II, we discussed how previous methods have approached this problem by using a variety of physical and statistical estimates. In this work, we take a different approach, and do not attempt to accurately learn a global model of the dynamics. Instead, we assume that we have access to only a weak prior model of the form $\tilde{p}(\mathbf{x}_t, \mathbf{u}_t, \mathbf{x}_{t+1})$, and estimate a local linear model of the dynamics under this prior. The local linear model is updated at each time step, which allows our method to gradually adapt to changing dynamics conditions and compensate for inaccuracies in the prior model.

IV. Model-Based Reinforcement Learning with Online Dynamics Adaptation

Our model-based reinforcement learning approach with online dynamics adaptation consists of using MPC (with iterative LQR) to repeatedly update the robot's policy under an evolving dynamics model. The dynamics model is linear time-varying, but is recomputed at each time step based on the recently observed states and actions, as well as the dynamics prior, which can take on one of a number of different representations and is trained on previous robot experience, which might even be from a different task. In this section, we describe how linear dynamics can be fitted under a dynamics prior, as well as our scheme for updating the dynamics online based on the robot's recent experience. We then summarize our model-based reinforcement learning algorithm.

A. Fitting Dynamics with Priors

In order to fit a linear model of the dynamics to a set of N samples $\{\mathbf{x}_i, \mathbf{u}_i, \mathbf{x}'_i\}$, we can simply use standard linear regression to determine $f_{\mathbf{x}}$, $f_{\mathbf{u}}$, and f_c. To make it more convenient to incorporate prior information, we will first reformulate this linear regression fit and view it as fitting a Gaussian model to the dataset $\{\mathbf{x}, \mathbf{u}, \mathbf{x}'\}$, and then conditioning this Gaussian to obtain $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$. While this is equivalent to linear regression, it allows us to easily incorporate a normal inverse-Wishart prior on this Gaussian in order to bring in prior information. Let $\hat{\Sigma}$ be the empirical covariance of our dataset, and let $\hat{\mu}$ be the empirical mean. The normal-inverse-Wishart prior is defined by prior parameters $\mathbf{\Phi}$, μ_0, m, and n_0. Under this prior, the maximum a posteriori estimates for the covariance Σ and mean μ are given by

$$\Sigma = \frac{\mathbf{\Phi} + N\hat{\Sigma} + \frac{Nm}{N+m}(\hat{\mu} - \mu_0)(\hat{\mu} - \mu_0)^{\mathrm{T}}}{N + n_0}$$
$$\mu = \frac{m\mu_0 + n_0\hat{\mu}}{m + n_0}. \tag{1}$$

Having obtained Σ and μ, we can obtain an estimate of the dynamics $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ by conditioning the distribution $\mathcal{N}(\mu, \Sigma)$ on $[\mathbf{x}_t; \mathbf{u}_t]$, which produces linear-Gaussian dynamics $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t) = \mathcal{N}(f_{\mathbf{x}t}\mathbf{x}_t + f_{\mathbf{u}t}\mathbf{u}_t + f_{ct}, \mathbf{F}_t)$, given by

$$\begin{aligned} f_{\mathbf{xu}} &= \Sigma^{-1}_{[\mathbf{xu},\mathbf{xu}]}\Sigma_{[\mathbf{xu},\mathbf{x}']} \\ f_c &= \mu_{[\mathbf{x}']} - f_{\mathbf{xu}}\mu_{[\mathbf{xu}]} \\ \mathbf{F} &= \Sigma_{[\mathbf{x}',\mathbf{x}']} - f_{\mathbf{xu}}\Sigma_{[\mathbf{xu},\mathbf{xu}]}f^T_{\mathbf{xu}} \end{aligned} \tag{2}$$

where the bracketed subscripts denote the submatrix corresponding to the specified elements. The parameters of the normal-inverse-Wishart prior are obtained from some prior model of the dynamics. We discuss how a variety of global dynamics model, such as Gaussian mixture models and neural networks, can be used to produce the prior parameters $\mathbf{\Phi}$, μ_0, m, and n_0 in Section V.

B. Online Estimation of Locally Linear Dynamics

In the batch setting, such as the one described in prior work [17], the empirical mean $\hat{\mu}$ and covariance $\hat{\Sigma}$ are estimated from a batch of previously collected data. In this work, we instead would like to update the model online during execution. To do this efficiently, we use an online estimate for $\hat{\mu}$ and $\hat{\Sigma}$. The mean can be estimated simply by using the following update:

$$\hat{\mu}_t \leftarrow \beta\hat{\mu}_{t-1} + (1-\beta)\mathbf{p}_t, \tag{3}$$

where $\mathbf{p}_t = [\mathbf{x}_{t-1}; \mathbf{u}_{t-1}; \mathbf{x}_t]$ is the tth observation and β is a discounting factor that causes the model to forget old data. For the covariance, first observe that, in the batch case, $\hat{\Sigma} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{p}_i\mathbf{p}_i^{\mathrm{T}} - \hat{\mu}\hat{\mu}^{\mathrm{T}}$. We can therefore estimate $\hat{\Sigma}_t$ as $\hat{\Sigma}_t = \Delta_t - \hat{\mu}_t\hat{\mu}_t^{\mathrm{T}}$, where Δ_t is the current estimate of $\frac{1}{t}\sum_{t'=1}^{t}\mathbf{p}_{t'}\mathbf{p}_{t'}^{\mathrm{T}}$, which we update according to:

$$\Delta_t \leftarrow \beta\Delta_{t-1} + (1-\beta)\mathbf{p}_t\mathbf{p}_t^{\mathrm{T}}. \tag{4}$$

These updates allow us to estimate the empirical mean $\hat{\mu}_t$ and covariance $\hat{\Sigma}_t$ at the tth step using recently observed state transitions. The posterior mean and covariance Σ and μ can then be recovered using the prior and the method described in the previous section. We initialize $\hat{\mu}_0$ and Δ_0 by fitting a single Gaussian to the data used for training the prior, in order to start with a reasonable estimate of the dynamics.

The value of β determines how quickly the algorithm "forgets" past experiences. When the true dynamics are nonlinear, as in most robotic manipulation tasks, we should forget

Algorithm 1 Model-based reinforcement learning with online adaptation

for time step $t = 1$ to T **do**
  Observe state $\mathbf{x}_t$
  Update $\hat{\mu}_t$ and Δ_t via Equations (3) and (4)
  Compute $\hat{\Sigma}_t = \Delta_t - \hat{\mu}_t \hat{\mu}_t^{\mathrm{T}}$
  Evaluate prior to obtain $\mathbf{\Phi}$, μ_0, m, and n_0 (see Section V)
  Update β and N as described in Equation (5)
  Compute μ and Σ via Equation (1)
  Compute $f_{\mathbf{x}t}$, $f_{\mathbf{u}t}$, f_{ct}, and $\mathbf{F}_t$ from μ and Σ via Equation (2)
  Run LQR to compute $\mathbf{K}_t$, $\mathbf{k}_t$, and $Q_{\mathbf{u},\mathbf{u}t}$
  Sample $\mathbf{u}_t$ from $\mathcal{N}(\hat{\mathbf{u}}_t + \mathbf{k}_t + \mathbf{K}_t(\mathbf{x}_t - \hat{\mathbf{x}}_t), Q^{-1}_{\mathbf{u},\mathbf{u}t})$
  Take action $\mathbf{u}_t$
end for

past experiences more quickly when the state is changing rapidly. However, when the robot becomes stuck in a difficult situation that requires a more accurate dynamics estimate, it must incorporate more data, which requires a larger value of β. The effective sample size N is also important, since it controls the relative strength of the prior. The true effective sample size is in fact inversely proportional to $1 - \beta$. We adaptively adjust both β and N based on the relative accuracy of the empirical and prior dynamics estimates. The intuition is that, when the prior is less effective, we should weight the empirical estimate more highly. We should also raise β, since the empirical estimate factors more highly in the dynamics, and therefore must use more of the past data to improve its accuracy. This intuition is also supported by the inverse relationship between $1 - \beta$ and N.

Specifically, we can form the prediction for the most recently observed transition from $(\mathbf{x}_{t-1}, \mathbf{u}_{t-1})$ to $\mathbf{x}_t$ using both the empirically estimated parameters $\hat{\Sigma}_t$ and $\hat{\mu}_t$ and the prior parameters $\frac{1}{n_0}\mathbf{\Phi}_t$ and μ_{0t}. We condition both $\mathcal{N}(\hat{\mu}_t, \hat{\Sigma}_t)$ and $\mathcal{N}(\mu_{0t}, \frac{1}{n_0}\mathbf{\Phi}_t)$ on $(\mathbf{x}_{t-1}, \mathbf{u}_{t-1})$ and predict the most probable value for the current state, which we denote $\hat{\mathbf{x}}_t$ for the prediction from the empirical parameters and $\bar{\mathbf{x}}_t$ for the prediction from the prior parameters. We then compute the ratio of the errors in these predictions, to compare whether the prior or empirical estimate is more accurate and update β and N:

$$\rho = \frac{\|\hat{\mathbf{x}}_t - \mathbf{x}_t\|^2}{\|\bar{\mathbf{x}}_t - \mathbf{x}_t\|^2} \qquad \beta = 1 - \eta_0 \rho \qquad N = \nu_0 / \rho, \tag{5}$$

where η_0 and ν_0 are hyperparameters of the algorithm. Note that, in these updates, N is inversely proportional to $1 - \beta$, as expected, and the proportionality constant is controlled by η_0 and ν_0, which we set as $\nu_0 = 1$ and $\eta_0 = 8$ in all of our experiments.

C. Algorithm Summary

The structure of our method is summarized in Algorithm 1. At each time step, the method observes the current state $\mathbf{x}_t$ and uses $[\mathbf{x}_{t-1}, \mathbf{u}_{t-1}, \mathbf{x}_t]$ to update the current estimate of the empirical mean and covariance $\hat{\mu}$ and $\hat{\Sigma}$ as described in the previous section. The method then evaluates the prior to obtain $\mathbf{\Phi}$, μ_0, m, and n_0, and updates β and N based on the accuracy of the empirical and prior state prediction. The empirical and prior estimates are then combined according to Equation (1) to construct the posterior mean and covariance, from which we can obtain an estimate for the current dynamics. These estimated dynamics are then used, together with a local second order expansion of the cost function, to optimize a new linear feedback policy using LQR. This feedback policy, given by $g(\mathbf{x}_t) = \hat{\mathbf{u}}_t + \mathbf{k}_t + \mathbf{K}_t(\mathbf{x}_t - \hat{\mathbf{x}}_t)$, can then be used to choose the next action $\mathbf{u}_t$.

In practice we often want to perform a small amount of exploration. For example, if the robot is attempting a peg insertion task, and the peg becomes jammed in the hole, simply applying the estimated optimal action repeatedly may be insufficient. The robot must "wiggle" its arm to figure out the contact dynamics. To that end, we add a small amount of Gaussian noise to the action. The amount of noise to add is determined by the Q-function obtained from LQR, by setting the covariance to be proportional to $Q^{-1}_{\mathbf{u},\mathbf{u}t}$. This choice of covariance is motivated by the observation that it produces a maximum entropy policy that properly trades off randomness and cost minimization, as discussed in prior work [19].

V. Neural Network Dynamics Priors

In this section, we discuss several possible choices for the dynamics prior, describe how the normal-inverse-Wishart prior parameters can be obtained from these priors, and go into detail on the particular neural network prior that we use in this work. The various choices for the priors are compared in our experimental evaluation in Section VI.

A. Gaussian and Gaussian Mixture Priors

As discussed in the previous section, our online estimate of the dynamics linearization makes use of a dynamics prior. The simplest prior can be obtained by fitting a Gaussian distribution to vectors $[\mathbf{x}; \mathbf{u}; \mathbf{x}']$ obtained from a large batch of prior data. If the mean and covariance of this prior data are given by $\bar{\mu}$ and $\bar{\Sigma}$, the prior is given by $\mathbf{\Phi} = n_0 \bar{\Sigma}$ and $\mu_0 = \bar{\mu}$, while n_0 and m should be set to the number of data points in the prior datasets. In practice, setting n_0 and m to 1 tends to produce substantially better results, since the online empirical mean and covariance are typically obtained from a much smaller number of samples.

A more sophisticated choice of prior explored in previous work is a Gaussian mixture model over vectors $[\mathbf{x}; \mathbf{u}; \mathbf{x}']$, which allows modeling of nonlinear dynamics [17]. Under this model, the state transition tuple is assumed to come from a distribution that depends on some hidden state h_i, which corresponds to the mixture element identity. In practice, this hidden state might correspond to the type of contact profile experienced by a robotic arm at step i. The prior is obtained by inferring the hidden state h_i for the latest tuple $[\mathbf{x}_{i-1}; \mathbf{u}_{i-1}; \mathbf{x}_i]$, and using the mean and covariance of the corresponding mixture element to obtain $\bar{\mu}$ and $\bar{\Sigma}$. The prior parameters can then be obtained as described above.

B. Neural Network Priors

The Gaussian prior is limited to representing globally linear dynamics, while the Gaussian mixture model can only represent a small number of locally linear modes, and has limited representational capacity in complex state spaces. Recent work has shown that neural networks can learn very good dynamics models for tasks ranging from helicopter flight [20] to cutting vegetables [12]. We therefore also evaluated the use of neural networks for representing the dynamics prior, by training a neural network to map from state-action tuples $[\mathbf{x}; \mathbf{u}]$ to the next state $\mathbf{x}'$.

In order to construct a dynamics prior from a neural network $\bar{f}([\mathbf{x}; \mathbf{u}])$ at the current state and action $[\mathbf{x}_i; \mathbf{u}_i]$, we first linearize to obtain

$$\bar{f}([\mathbf{x}; \mathbf{u}]) \approx \bar{f}([\mathbf{x}_i; \mathbf{u}_i]) + \frac{d\bar{f}}{d[\mathbf{x}; \mathbf{u}]}^{\mathrm{T}} ([\mathbf{x}; \mathbf{u}] - [\mathbf{x}_i; \mathbf{u}_i]) \,.$$

From this linearization, we can construct a local prior mean and covariance according to

$$\bar{\mu} = \begin{bmatrix} [\mathbf{x}_i; \mathbf{u}_i] \\ \bar{f}([\mathbf{x}_i; \mathbf{u}_i]) \end{bmatrix}$$

$$\bar{\Sigma} = \begin{bmatrix} \bar{\Sigma}_{\mathbf{xu},\mathbf{xu}} & \frac{d\bar{f}}{d[\mathbf{x};\mathbf{u}]}^{\mathrm{T}} \bar{\Sigma}_{\mathbf{xu},\mathbf{xu}} \\ \bar{\Sigma}_{\mathbf{xu},\mathbf{xu}} \frac{d\bar{f}}{d[\mathbf{x};\mathbf{u}]} & \frac{d\bar{f}}{d[\mathbf{x};\mathbf{u}]}^{\mathrm{T}} \bar{\Sigma}_{\mathbf{xu},\mathbf{xu}} \frac{d\bar{f}}{d[\mathbf{x};\mathbf{u}]} + \bar{\Sigma}_{\mathbf{x}',\mathbf{x}'} \end{bmatrix} .$$

The prior state-action covariance $\bar{\Sigma}_{\mathbf{xu},\mathbf{xu}}$ determines the strength of the prior, and we set it to $\alpha\mathbf{I}$, where α is a free parameter. The conditional covariance $\bar{\Sigma}_{\mathbf{x}',\mathbf{x}'}$ can be obtained from the empirical covariance between the network predictions $\bar{f}([\mathbf{x}; \mathbf{u}])$ and the target next states $\mathbf{x}'$ in the training dataset. Using the above $\bar{\mu}$ and $\bar{\Sigma}$, the normal-inverse-Wishart prior parameters can be obtained in the same way for the Gaussian and mixture of Gaussians priors.

C. Neural Network Architectures

In order to represent the dynamics prior, we used a neural networks with two hidden layers, each with rectified linear units given by $z = \max(a, 0)$. The first hidden layer consists of 60 hidden units, and the second consists of 40. We evaluated two network architectures, both of which are illustrated in Figure 2. The first takes the current state $\mathbf{x}_t$ and current action $\mathbf{u}_t$ as input, and predicts accelerations. These accelerations are then integrated via a semi-implicit integration rule to obtain a prediction for $\mathbf{x}_{t+1}$, and the network is optimized to minimize the error in predicting the entire next state $\mathbf{x}_{t+1}$.

Our second architecture is identical to the first, but instead takes as input the current and previous states and actions $(\mathbf{x}_{t-1}, \mathbf{u}_{t-1}, \mathbf{x}_t, \mathbf{u}_t)$. This additional temporal context is very important when the dynamics prior is trained on other tasks, especially tasks that involve contacts with objects in the environment. Since the state $\mathbf{x}_t$ only includes the configuration of the robot, it does not model the geometry of the scene, which might change across tasks. The state is therefore not Markovian across tasks, and a network without additional context cannot accurately predict the next state $\mathbf{x}_{t+1}$ from only the current state $\mathbf{x}_t$ and action $\mathbf{u}_t$. We found

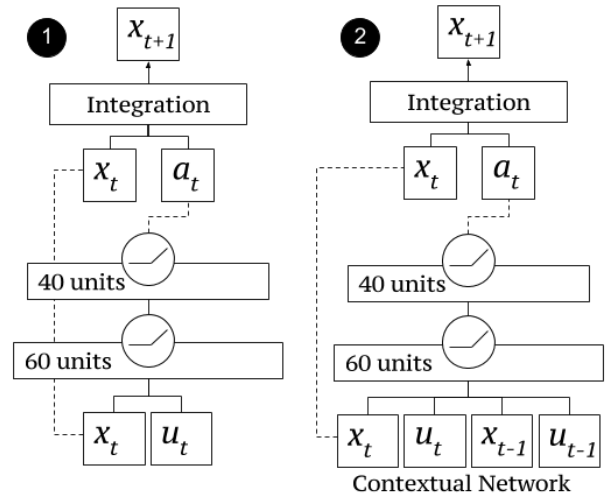

Fig. 2: Diagram of the neural network architectures used in our experiments. We found that using a short temporal context as input, as shown in network (2), improved the results for manipulation tasks that involved contact dynamics. Both networks produce accelerations which are used to predict the next state.

that the network without temporal context tried to explain contacts by exploiting certain regularities. For example, when the end-effector stopped after hitting an object, the network assumed that a hard contact had occurred and there is an impassable obstruction in the way. With a temporal context that included the previous state and action, the network was able to determine whether or not a contact was happening by comparing the previous applied joint torques to the acceleration actually experienced by the robot.

D. Prior Training Data

The neural network prior must be trained on previous interaction data in order to provide a helpful prior model of the system dynamics. However, because the neural network only acts as a prior, it can be trained on previous interaction data from different tasks. This allows our method to perform one-shot learning of new manipulation tasks, using no prior data from that task itself. In practice, the amount of data required to learn an effective prior is considerable, so we used data collected from a variety of sources to provide a sufficiently diverse dataset. The total training set for the physical robot experiments had 6.6 hours of data, collected at 20 Hz. For each individual task, we excluded the data collected for that task when training the prior, which reduced the effective dataset by about 15%. This corresponds to a type of holdout cross-validation. Likewise, in the simulated experiments, we collected approximately 12 simulation hours of data at 20Hz across 4 different tasks, and excluded the data from the task being executed from the training set.

The dataset consisted of trials from the various evaluation tasks (workbench, gears, airplane, car, and ring tasks for the physical system and peg insertion and stacking for the simulated system), which are described in the next section, as well as random motion of the arm in free space. For each task, data was collected using a previous reinforcement

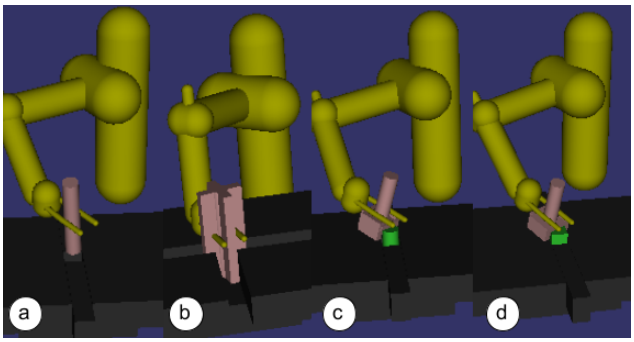

Fig. 3: Simulated tasks used for evaluation: (a) Cylindrical peg insertion (b) Cross-shaped peg insertion (c) Stacking over a cylindrical peg (d) Stacking over a square-shaped peg

learning method [21], though in practice this data could have also been generated using prior experience from the method presented in this paper, human demonstrations, or any other method that provides good coverage of interesting states (e.g. states that involve contact with objects in the world). Although the size of this dataset is considerable, the resulting prior, when combined with our online adaptation method, can generalize effectively to other tasks, making it quite universal.

VI. Experimental Evaluation

A. Experimental Setup

We evaluated our method on a range of manipulation tasks using a physical PR2 mobile manipulator and a simulated PR2 arm using the MuJoCo simulator [11]. The simulated tasks provide a large-scale comparison between methods, and the physical tasks demonstrate effectiveness on a real system.

On the physical system, our method was used to control the right arm of the robot, while the left arm was used to brace objects for two-object manipulation and assembly tasks. We evaluated the method by inserting a toy nail into a tool bench, including a high-friction version of the task designed to make it more difficult, placing wooden rings onto pegs, putting together parts of a gear assembly, assembling a toy car and airplane, and stacking blocks. These tasks are shown in Figure 4. Our simulated tasks consisted of a pair of peg insertion tasks with different shaped pegs, and a pair of peg stacking tasks, where the robot needs to fit a piece with a square-shaped opening over a cylindrical or square shaped peg. These are shown in Figure 3. A trial was defined to be a failure if the distance to a target pose was not sufficiently small at the end of a time threshold of 10 seconds.

The state space consisted of several measurements from the active right arm of the robot: joint angles, joint angular velocities, the pose of the end effector encoded as 3 Cartesian points, and the velocities of those 3 points. This amounted to a 32-dimensional state space. The controller operated at 20 Hz with an MPC horizon of 15 timesteps (0.75 seconds).

The cost function for each task depended on the distance d between the current pose of the end-effector and a target pose required for successful completion. For example, in the case of the toy nail task, the target pose involved positioning the gripper such that the nail was successfully inserted into a hole in the toy tool bench. The shape of this cost follows prior work [21], and is given by $r_\ell(d) = wd^2 + v\log(d^2+\alpha)$, where $\alpha = 10^{-5}$, $w = 1.0$, and $v = 0.01$. This type of shape encourages the robot to accurately place the object in precisely the desired configuration. In addition, we placed a quadratic penalty on the magnitude of the torques.

As described in the previous section, the neural network dynamics prior was trained using data collected from all of the other tasks, but no data from the task being tested. This corresponds to a hold-one-out cross-validation scheme.

B. Comparisons

We evaluated our adaptive method with two neural network priors: network #1 in Figure 2, which uses only the current state $\mathbf{x}_t$ and action $\mathbf{u}_t$ to predict the next state $\mathbf{x}_{t+1}$, and network #2, which also uses the preceding state and action. In addition, we also compared to a number of baselines. The non-adaptive baseline used network #2, which was the better of the two neural network priors, to directly plan actions using MPC, which most closely resembles the structure of a previously proposed neural network-based MPC method [12]. The Gaussian process baselines used a Gaussian process either as a prior for the adaptive approach, or as the model (without adaption) for MPC, which most closely reflects a range of recent Gaussian process model-based reinforcement learning algorithms [7], [22]. Since the Gaussian process is a nonparametric model, we were unable to use the same amount of data with this model and maintain real time performance. We therefore subsampled the data to the largest size that still permitted online MPC (approximately 10000 training points, which was about 5% of the total training set for the physical robot). Finally, the regularized model substituted a global linear model instead of the neural network dynamics prior.

We show the success rates for each method on the simulated robot in Table I and on the physical robot in Tables II (for the standard and high-friction variant of the toolbench and toy nail task, as well as for the wooden ring on a peg task) and III (for the block, gears, airplane, and car assembly tasks). Note that each of the runs used to evaluate each method was performed separately, with no information retained between runs. The adaptive method outperformed the non-adaptive variants, indicating the importance of online adaptation for models trained on other tasks. The neural network prior also achieved the best overall performance, particularly when using the preceding state and action as context, although even the simple least-squares prior was able to accomplish some of the simpler tasks.

We also show the success rates for our adaptive method with network #2 on each of the other tasks. These results show that our method was able to succeed on a wide range of challenging manipulation tasks on the first try, without using any data from that task to train the dynamics prior.

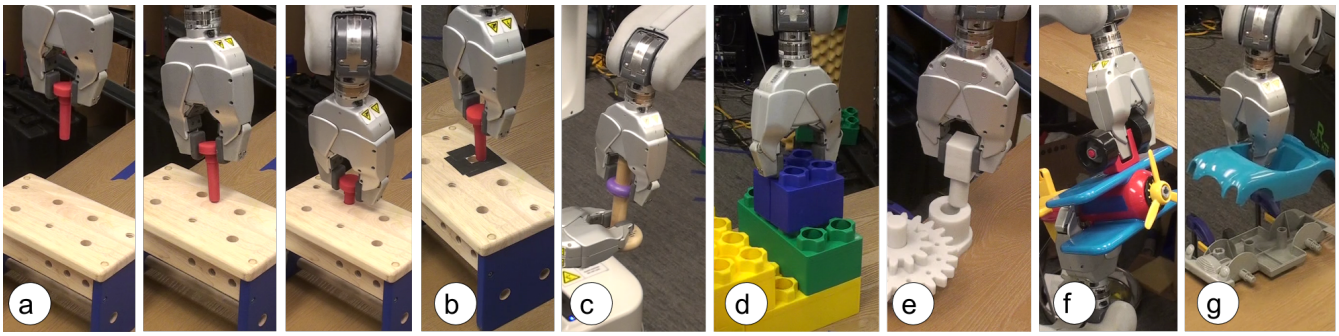

Fig. 4: Tasks used in our evaluation: (a) inserting a toy nail into a toolbench, (b) inserting the nail with a high-friction surface to increase difficulty, (c) placing a wooden ring on a tight-fitting peg, (d) stacking toy blocks, (e) putting together part of a gear assembly, (f) assembling a toy airplane and (g) a toy car.

Task	Ins, Cylinder (a)	Ins, Cross (b)	Stack, Cylinder (c)	Stack, Square (d)	Total
Adaptation + regularization	10/25	5/25	18/25	10/25	43/100
Adaptation + GMM prior	10/25	6/25	**25/25**	18/25	59/100
Contextual network (#2)	8/25	1/25	19/25	12/25	40/100
Adaptation + network #1	14/25	8/25	20/25	14/25	56/100
Adaptation + Contextual network (#2) (**full method**)	**19/25**	**11/25**	**25/25**	**20/25**	**75/100**

TABLE I: Success rate of our method on simulated tasks, as well as comparisons to a range of baselines representative of prior methods on three of the tasks.

Task	Toolbench (a)	W/ friction (b)	Ring (c)	Total
Adaptation + regularization	2/5	0/5	**4/5**	6/15
Adaptation + GMM prior	0/5	1/5	2/5	3/15
Gaussian process	0/5	0/5	0/5	0/15
Adaptation + gaussian process	1/5	0/5	0/5	1/15
Contextual network (#2)	3/5	3/5	**4/5**	10/15
Adaptation + network #1	4/5	0/5	3/5	7/15
Adaptation + Contextual network (#2) (**full method**)	**5/5**	**4/5**	**4/5**	**13/15**

TABLE II: Success rate of our method on each physical task. Note that our approach regularly succeeds at the task despite only using training data from other tasks.

Task	Success Rate
Block (d)	4/5
Gears (e)	4/5
Airplane (f)	3/5
Car (g)	3/5

TABLE III: Success rates on additional tasks using adaptation with network #2 only.

Task	Block			Toolbench			Ring		
Target position error	0.5 cm	1.0 cm	1.5 cm	0.5 cm	1.0 cm	1.5 cm	0.5 cm	1.0 cm	1.5 cm
Contextual network (#2)	2/5	0/5	0/5	4/5	2/5	0/5	4/5	4/5	1/5
Adaptation + contextual network (#2) (**full method**)	4/5	1/5	0/5	3/5	3/5	1/5	4/5	3/5	0/5

TABLE IV: Robustness results in the presence of target position errors. The target was intentionally offset from the true position of the target by the indicated amount in random direction along the horizontal plane.

C. Robustness

We also evaluated the robustness of our method to observation errors. In these experiments, the target position in the cost function was intentionally corrupted by fixed magnitude errors. In real-world scenarios, these errors might stem from imperfect observations produced, for example, by a vision system. The results of the robustness experiments on two of the tasks are presented in Table IV. For comparison, we also include robustness results for the neural network only (using network #2), without adaptation. The ring was the easiest of the three tasks, since the rounded top of the peg provides some tolerance, while the block was the hardest. Note that adaptation is particularly helpful on the harder tasks.

D. Qualitative Results and Conclusions

Our experimental results show that our model-based reinforcement learning algorithm with online adaptation can achieve a range of challenging manipulation tasks on the first attempt. Furthermore, our comparison of the various prior models shows that a neural network prior with a short temporal context achieved the best results, though no prior model by itself was as successful as the corresponding adaptive variant. Videos of our tasks, which can be viewed in the supplementary material and on the project website, (`http://rll.berkeley.edu/iros2016onlinecontrol/index.html`) show that the resulting behaviors can take some time to succeed at the task (though never more than 20 seconds). Much of this time is spent exploring unfamiliar dynamical modes, for example after an unexpected contact. This kind of exploration reflects a very natural strategy for performing unfamiliar tasks: probing the object until progress is made in the desired direction, while gradually acquiring a more accurate model. It is precisely this probing and gradual self-improvement that is key to the robustness and effectiveness of this approach.

Acknowledgements

This research was funded in part by the Army Research Office through the MAST program, and by Darpa under Award #N66001-15-2-4047.

VII. Discussion and Future Work

In this paper, we presented a model-based reinforcement learning algorithm that uses online adaptation of its dynamics model combined with a coarse prior model obtained from experience on other prior tasks. We use an MPC method based on iterative LQR to choose the actions under the current adapted model, which is a local linear approximation to the nonlinear dynamics of the system. Although this linear model is simple, our method is able to perform complex manipulation tasks in highly nonlinear systems by adapting it to the most recent experience and re-planning. The coarse prior model is therefore not required to accurately model the true dynamics of the system, and only provide a guess that is refined through online adaptation. Our experiments show that even a fully linear prior model can be used for some behaviors, while a more sophisticated neural network prior allows our method to complete more complex tasks.

We believe that combining online model adaptation with prior experience by means of a dynamics prior is a powerful idea because it allows our method to succeed even with unexpected environmental variation and inaccurate prior models. Furthermore, by using rich function approximators such as large neural networks to distill prior experience into a concise parametric representation, the robot can progressively become more proficient at acquiring new skills as its experience grows, similar to how a person can quickly learn new skills by drawing on past experiences. However, in contrast to humans, robots can pool their collective experience and use the combined data to train shared dynamics priors. Exploring this type of multi-robot learning, with shared priors but individual adaptation, is an exciting direction for future work.

While in our evaluations, we aggregated data from all other tasks to train the dynamics model, we may want to only use related tasks to avoid negative transfer. This may be accomplished by clustering or grouping the prior tasks.

Although we demonstrate that our approach can complete a variety of challenging manipulation tasks, a limitation of this method is that we require a full, Markovian state of the system, which is needed to perform MPC using iterative LQR. This assumption is not unusual for MPC-based methods, but can be limiting in the context of robotic manipulation. One approach for addressing this is to learn a latent state representation from raw sensory input, as proposed in recent work [23], [24], [25], and perform MPC on this learned representation with online dynamics adaptation.

Another avenue for future work is to combine other kinds of prior information with online updates. For example, a prior policy might be constructed from experience of related tasks, and refined in a similar fashion as the dynamics adaptation. This could allow our relatively short-horizon MPC procedure to acquire longer lookahead through the use of the prior policy, and allow the use of rich sensory data as demonstrated in recent work on policy search with neural networks [26].

References

[1] M. Deisenroth, G. Neumann, and J. Peters, "A survey on policy search for robotics," *Foundations and Trends in Robotics*, vol. 2, no. 1-2, pp. 1–142, 2013.

[2] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *International Journal of Robotic Research*, vol. 32, no. 11, pp. 1238–1274, 2013.

[3] D. Nguyen-Tuong and J. Peters, "Using model knowledge for learning inverse dynamics," in *International Conference on Robotics and Automation (ICRA)*, 2010.

[4] M. Cutler and J. P. How, "Efficient reinforcement learning for robots using informative simulated priors," in *International Conference on Robotics and Automation (ICRA)*, 2015.

[5] Y. Wu and Y. Demiris, "Towards one shot learning by imitation for humanoid robots," in *International Conference on Robotics and Automation (ICRA)*, 2010.

[6] J. Peters and S. Schaal, "Reinforcement learning of motor skills with policy gradients," *Neural Networks*, vol. 21, no. 4, pp. 682–697, 2008.

[7] M. Deisenroth and C. Rasmussen, "PILCO: a model-based and data-efficient approach to policy search," in *International Conference on Machine Learning (ICML)*, 2011.

[8] J. Boedecker, J. T. Springenberg, J. Wulfing, and M. Riedmiller, "Approximate real-time optimal control based on sparse Gaussian process models," in *Adaptive Dynamic Programming and Reinforcement Learning (ADPRL)*, 2014.

[9] K. J. Astrom and B. Wittenmark, *Adaptive Control*, 2nd ed. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 1994.

[10] D. Windate, N. D. Goodman, D. M. Roy, L. P. Kaelbling, and J. B. Tenenbaum, "Bayesian policy search with policy priors," *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence (IJCAI)*, 2011.

[11] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012.

[12] I. Lenz, R. Knepper, and A. Saxena, "DeepMPC: Learning deep latent features for model predictive control," in *Robotics: Science and Systems (RSS)*, 2015.

[13] H. Fukushima, T. Kim, and T. Sugie, "Adaptive model predictive control for a class of constrained linear systems based on the comparison model," *Automatica*, vol. 43, no. 2, pp. 301–308, Feb. 2007.

[14] A. Aswani, P. Bouffard, and C. Tomlin, "Extensions of learning-based model predictive control for real-time application to a quadrotor helicopter," in *American Control Conference (ACC)*, 2012.

[15] G. Chowdhary, M. Mühlegg, J. How, and F. Holzapfel, "Concurrent learning adaptive model predictive control," in *Advances in Aerospace Guidance, Navigation and Control*. Springer Berlin Heidelberg, 2013.

[16] W. Li and E. Todorov, "Iterative linear quadratic regulator design for nonlinear biological movement systems," in *ICINCO (1)*, 2004, pp. 222–229.

[17] S. Levine and P. Abbeel, "Learning neural network policies with guided policy search under unknown dynamics," in *Advances in Neural Information Processing Systems (NIPS)*, 2014.

[18] Y. Tassa, T. Erez, and E. Todorov, "Synthesis and stabilization of complex behaviors through online trajectory optimization," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012.

[19] S. Levine and V. Koltun, "Guided policy search," in *International Conference on Machine Learning (ICML)*, 2013.

[20] A. Punjani and P. Abbeel, "Deep learning helicopter dynamics models," in *International Conference on Robotics and Automation (ICRA)*, 2015.

[21] S. Levine, N. Wagener, and P. Abbeel, "Learning contact-rich manipulation skills with guided policy search," in *International Conference on Robotics and Automation (ICRA)*, 2015.

[22] Y. Pan and E. Theodorou, "Probabilistic differential dynamic programming," in *Neural Information Processing Systems (NIPS)*, 2014.

[23] M. Riedmiller, S. Lange, and A. Voigtlaender, "Autonomous reinforcement learning on raw visual input data in a real world application," in *International Joint Conference on Neural Networks*, 2012.

[24] M. Watter, J. T. Springenberg, J. Boedecker, and M. Riedmiller, "Embed to control: A locally linear latent dynamics model for control from raw images," in *Advances in Neural Information Processing Systems (NIPS)*, 2015.

[25] C. Finn, X. Y. Tan, Y. Duan, T. Darrell, S. Levine, and P. Abbeel, "Deep spatial autoencoders for visuomotor learning," *International Conference on Robotics and Automation (ICRA)*, 2016.

[26] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *Journal of Machine Learning Research (JMLR)*, 2016.